

基於 Slurm 與 Kubernetes 架構下 AI 伺服器 GPU 工作負載智慧排程技術之研究

ABC DEF
國立 XX 大學

{s11259033, s11259043}@stumail.nutn.edu.tw

摘要

本研究以一般大學實驗室共享 GPU 主機常見的資源閒置、GPU 利用率不足與批次工作管理困難為背景，探討如何在容器化環境中建立兼具彈性與穩定性的 AI 工作負載排程平台。旨在整合 Slurm 的批次排程能力與 Kubernetes 的容器編排及彈性擴縮能力，讓使用者能以熟悉流程提交工作，同時由系統自動完成資源啟動、分配與監控。本專題提出以 Slurm-on-Kubernetes 為核心的共享 GPU 排程架構，並導入 MPS 硬體資源切分、節點自動擴縮、監控介面與深度強化學習排程插件等策略，以支援多工作共享 GPU 及智慧化資源放置。

關鍵詞：GPU 資源排程、Kubernetes、Slurm、深度強化學習。

Abstract

Large-scale AI workloads have rapidly increased the demand for GPU scheduling. Existing Slurm schedulers mainly allocate entire GPUs and rarely consider heterogeneous GPUs together with NVIDIA MPS, leading to poor resource utilization and long queueing delays. Existing heuristic scheduling policies such as FCFS and Backfill cannot dynamically adapt to workload characteristics, while existing DRL schedulers rarely support heterogeneous GPU placement and MPS allocation in real Slurm environments. This paper proposes XXXXX. Compared with FCFS and Backfill, our framework improves ...

Keywords: GPU Resource Scheduling, Kubernetes, Slurm, Deep Reinforcement Learning.

1. 緒論

Slurm Workload Manager 是目前高效能運算叢集最廣泛使用的工作排程系統之一。它提供完整的工作提交、佇列管理、資源限額與帳務管理功能，並支援 GPU 資源分配。其以「節點」為基礎的資源模型非常適合研究工作環境。然而 Slurm 原生設計以固定實體節點為主，對彈性擴縮與容器化整合的支援相對有限。而 Kubernetes 是目前最主流的容器編排平台，能夠自動管理容器的部署、擴縮與健康監控。Kubernetes 的 Cluster Autoscaler 可根據工作負載自動增減節點，非常適合需要彈性資源的場景。然而 Kubernetes 原生的排程器 (kube-scheduler) 以服務導向設計為主，對批次工作、GPU 共享與研究工作特有的排程需求支援不足。因此有多項研究嘗試將 Slurm 與 Kubernetes 整合，

以結合兩者優點。在 Slurm 與 Kubernetes 整合方面，既有研究已指出 HPC 與 cloud-native 系統的匯流可以同時取得批次排程語意與雲端彈性管理能力；實作層面也有 Slinky 等 Slurm-on-Kubernetes 方向的工具，以及 AWS ParallelCluster 等雲端 HPC 叢集部署方案，顯示此類架構已具有實務需求與發展基礎。

2. 相關研究

2.1 GPU 叢集排程、分析與資源共享

近期研究聚焦於單節點內或單一叢集模型上的動態資源調度本身。Wang 等人的 DCUDA [19] 針對單節點多 GPU 情境，設計了一套輕量級核心／記憶體使用率監控機制，搭配近乎零開銷的「執行中」CUDA 應用即時遷移，將 GPU 過載時間平均降低 78.3%、一般工作執行時間降低 42.1%（記憶體密集型工作最高 67%）。Sedighi 等人 [20] 則在 Alibaba 的 cluster-trace-gpu 生產工作負載軌跡之上，提出結合硬體與軟體分割的公平且需求感知動態資源配置演算法，於模擬環境中將 GPU 資源使用量降低達 88%。這兩項工作皆聚焦「資源配置本身如何隨工作負載動態調整」（即時遷移／再分割），評估分別侷限於單一多 GPU 節點與純模擬 trace 重放，並未涉及與批次排程器（如 Slurm）的整合或真實異質叢集上的統計驗證；本研究的排程決策則作用於 Slurm 的工作提交路徑之上，決定「哪個工作該去哪張卡的哪個 MPS 槽」，並在真實異質叢集上以失效安全與抗漂移統計方法落地檢驗，兩者的問題設定屬於不同但互補的抽象層次。

2.2 強化學習排程

Lin 等人 [23] 提出 UXP-RL：一個以 DQN 為核心、涵蓋前處理／訓練／推論三類任務、可部署為集中式或分散式排程器、並跨雲／邊／霧三層架構運作的 CPU-GPU 任務排程演算法。其於模擬環境中，集中式排程器將平均週轉時間相較 SJF／FCFS 與 TYPE 啟發式（依 GPU 需求高低分類任務）分別降低 57.81%、57.28% 與 27.66%；分散式排程器則因能將長訓練工作卸載至雲端而釋放邊／霧資源，把推論任務週轉時間相較集中式再降低 89.07%。同年 Zhang 等人的 KIS-S [25] 以 PPO 訓練一個 GPU-aware 的 Kubernetes 推論自動擴縮策略 (KIScaler)，完全於自建模擬器 (KISim) 中訓練後即零樣本部署，於多種流量情境下平均獎勵提升 75.2%、p95 延遲相較 CPU 基準降低最多 6.7

倍；其問題設定是調整副本數的自動擴縮，而非本研究的工作放置排程。Wu 等人的 DRR [26]則針對 GPU 共享叢集因分享機制、工作異質性與非同步生命週期造成的碎片化問題，以模仿學習從既有啟發式暖啟動一個深度強化學習去碎片化排程器，並輔以多尺度策略最佳化平衡探索與利用；其同時於實體 Kubernetes 測試床與大規模模擬叢集上驗證，平均碎片率降低 50%，是本節所列中少數同時涉及真實 Kubernetes 部署的學習式排程器。

2.3 異質與邊緣 GPU 排程

Tsenos 與 Kalogeraki 針對缺乏原生虛擬化支援的邊緣 GPU（如 RTX 4090、GTX 1080Ti 等消費級卡）提出一套硬體無關的時空共享機制：為每個行程建立 cgroup、動態調整其 duty cycle 來實現優先權與截止期限式排程，且無需修改工作負載原始碼即可整合進 TensorFlow、PyTorch、FFmpeg 等既有框架。而 Majeed 等人則以系統性文獻回顧整理 NVIDIA Jetson AGX 系列邊緣 SoC（CPU+GPU+深度學習加速器 DLA+可程式視覺加速器 PVA+視訊影像合成器 VIC）上的 DNN 排程器，區分規則式（如 Jedi、CP-CNN、Herald、H2H、HaX-CoNN）與最佳化式（線性規劃、AxoNN、遺傳演算法、以 Z3 求解器動態重排的 D-HaX-CoNN）兩大類，並整理其記憶體競爭、跨加速器轉移成本與靜態／動態排程的權衡。其排程粒度是單一 DNN 模型內的層級（將個別網路層指派給不同硬體加速器）。

3. 研究目的與系統架構

3.1 研究目的

目標是在異質 GPU 與 NVIDIA MPS 環境中，學習一個能同時決定 GPU placement 與 MPS allocation 的排程策略，以降低平均 JCT、尾部 JCT，同時提升 GPU 利用率。更具體而言，本研究要回答兩個問題。第一，DRL 是否能在模擬環境中學到比固定規則更好的 GPU+MPS 排程策略。第二，學習式策略在真實環境中是否能勝過 FCFS、Backfill、與啟發式策略。

本研究將異質 GPU+MPS 排程建模為馬可夫決策過程。每次有工作可排程時，代理人觀察叢集狀態，選擇一個 GPU 與 MPS fraction，並在工作完成後根據 JCT 與 GPU 利用率取得 reward。

狀態包含四類資訊：

- Job features：工作類型、預估 runtime、VRAM 需求、MPS 需求與等待時間。

- GPU features：GPU 型號、GPU 利用率、SM 利用率、記憶體用量、可用 MPS 資源、目前共置工作數。
- Queue features：佇列長度、前 K 個工作的需求、等待時間分布與到達率。
- History features：近期完成工作 JCT、slow-down 與各 GPU 的負載變化。

動作是「選擇佇列前 K 個工作中的哪一個」與「將其放置到哪張 GPU」的聯合離散決策，若某放置的 GPU 剩餘 MPS 資源不足以容納該工作的請求，動作遮罩會在推論前屏蔽該選項，避免產生不可執行決策。在本研究的 2 GPU 實驗平台中，動作空間總共有 8 個 actions。

Reward 的設計目標是降低使用者感受到的等待與完成時間，同時提高 GPU 使用效率，故設定為： $Reward = w_1 \times GPU_Util - w_2 \times JCT$ 。其中 JCT 代表工作完成時間，GPU_Util 代表資源使用效率。此 reward 設計讓 DRL 不只最佳化單一工作，而是學習長期排程結果。

3.2 系統架構

使用者提交工作後，Slurm 透過 job submission hook 呼叫自訂排程器，接著根據系統狀態資訊輸出資源分配決策。在工作執行期間，監控服務收集 GPU 利用率、記憶體用量、JCT 等資訊，將資料收集起來以供後續訓練或 RLDP (Reinforcement Learning with Prior Data) [24] 微調使用。

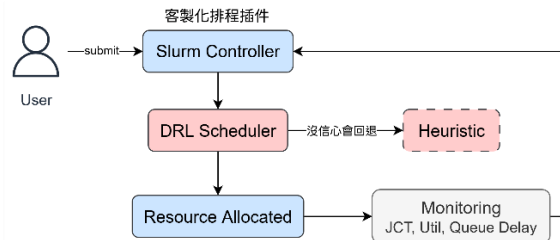


圖 1 工作提交到排程決策流程

本研究使用 Kubernetes (k3s) 部署 Slurm controller、worker、RL scheduler、monitoring service 與相關容器 [10]。Kubernetes 在本文中不負責排程決策；它只提供容器化部署、服務健康檢查、網路與生命週期管理。

4. 排程技術

4.1 Slurm 內建排程演算法

以 Slurm 原生排程能力作為穩定基線，而非重寫排程核心。例如 backfill 允許在不延後高優先權工作的前提下，讓資源需求較小、執行時間較短的工作提前插隊執行，緩解大工作長期佔用資源

造成的閒置，此外也納入更保守的 FCFS 作為下界對照，用以檢驗啟發式與學習式策略相對 Slurm 開箱即用能力是否確有改善。

4.2 啟發式排程策略

以加權線性組合公式計算工作優先級，分數越高代表該工作越值得優先排程。三個因子及對應權重說明如下表 1。

表 1 啟發式因子與係數定義

因子	係數	定義
MPS	0.40	$mps_req/100 \in [0, 1]$ 。沒給=1.0，超過=0.0。
VRAM	0.20	$(1 - (\text{fit_tier} - \text{req})) / \text{max_tier} \in [0, 1]$ 。依 job 需求選最小可用 VRAM tier，沒給=0.5。
Frag	0.20	$4(x - 1)$, $x = mps_req/100$ 。mps_req=0 或滿載 $\rightarrow 0.0$ ；mps_req=50% $\rightarrow$ 最高懲罰 ~ 1.0 。

4.3 深度強化學習策略

本研究比較以下 DRL 方法：

- Discrete SAC：將 Soft Actor-Critic 延伸到離散動作空間，以高斯策略搭配 reparameterization trick 取樣；Christodoulou [6] 提出的離散動作變體則以 categorical 策略取代高斯策略、以期望而非取樣估計熵項，使最大熵框架得以套用於本研究這類離散問題。
- RDSAC-mean：以分布式 critic 建模回報分布，但 actor 主要依平均 reward 做出決策 [7][16]。
- RDSAC-cvar：在 RDSAC 上加入 CVaR 風險敏感目標，使策略更重視尾端 JCT。
- RLPD：以真實資料對模擬訓練出的模型進行微調，縮小 sim-to-real 差距 [8]。本研究忠實採用原論文核心配方，每個批次對稱取樣 50% sim 先驗 + 50% 真實資料，但訓練機制為離線，非原論文的真線上更新。

惟本研究為離散動作、雙頭 IQN 的自組版本，是離散 SAC、IQN 分布式評論家與 CVaR 風險量度的組合，並非連續版本的完全複製。

5. 實驗與評估方法

5.1 實驗環境與訓練

本研究使用一個小規模異質 GPU 實機叢集進

行部署與評估，相關環境如表 2 所示。訓練資料集使用 Alibaba GPU Trace 與 Microsoft Philly Trace，前者用於參考生產 MLaaS 工作的到達率、工作長度與資源需求分布 [4]，後者用於參考多租戶 GPU training workload 的佇列與 JCT 特性 [1]。

表 2 實驗環境列表

項目	設定
GPU	NVIDIA RTX 4070、RTX 3080
部署平台	Kubernetes/k3s
GPU sharing	MPS fraction 為 25%、50%、75%、100%
系統	Ubuntu24.04 LTS、NVIDIA driver
監控	GPU utilization、SM utilization、memory usage、job event、queue delay

直接在實際環境從頭訓練的話，強化學習需要數十萬到數百萬個 transition，真實叢集一個編排決策對應一個跑數分鐘至數小時的任務，湊滿樣本要等數月，因此採 sim-to-real 兩段式：

- (1) 在模擬環境大量訓練，產出基本模型
- (2) 上線部署，記錄真實叢集 (observation, action, reward) 資料
- (3) RLPD 用真實資料把基本模型微調成真實環境策略

深度強化學習的訓練參數如下：

表 3 訓練參數設定

項目	數值
優化器／學習率	Adam／ 3×10^{-4}
隱藏層 (MLP trunk)	(256, 256) + LayerNorm
batch size / UTD ratio	256 / 4
n-step return	10
風險目標 / tail mass β	CVaR / 0.25 (RDSAC-cvar 臂)
溫度 α	自動調整 (init 0.1)
Reward	多目標 ($w_{jct}=1.0$, $w_{util}=0.05$)
訓練長度	1×10^5 curriculum env-steps

5.2 實機評估結果

在 BERT 推論、ResNet 訓練、Qwen 微調與矩陣運算混合工作負載進行評估，數量占比分別為 30%、30%、30% 和 10%，評估結果如表 4 所示。可以發現 RLPD 在平均工作完成時間(JCT)比其他方法好，而 RDSAC-cvar 在 DRL 策略中有優化到尾部。

表 4 評估結果

	JCT	P99	GPU 利用率
FCFS	137.0	241.2	8.60
Backfill	136.2	255.6	8.68
啟發式	125.2	433.5	8.25
SAC	126.2	471.2	8.00
RDSAC -mean	142.3	469.7	8.59
RDSAC -cvar	130.5	395.0	8.32
RLPD	110.7	418.0	7.45

6. 結論與未來展望

6.1 結論

相較傳統只選 GPU 或只做固定規則的方法，本研究將異質 GPU、MPS 配額、工作特徵、佇列狀態與回饋訊號整合成一個可學習的排程框架。結果顯示大部分學習式策略在平均工作完成時間比 Slurm 傳統排程好，惟本研究在實機評估是小規模實驗室環境，在較大型的叢集的效能有待驗證。整體而言，在異質 GPU + MPS 排程中，策略效益高度依賴工作負載、硬體規模與底層資源分配後端。

6.2 未來展望

- (1) 擴展至更大、更高競爭的叢集以檢驗「價值隨規模浮現」假設。
- (2) MIG + MPS fraction 混合 partition：同時納入硬體級隔離與軟體級共享，建立更完整的 GPU sharing action space。
- (3) Energy-aware scheduling：將功耗、能效與碳排納入 reward，使排程器不只最佳化效能，也最佳化能源效率。

參考文獻

按照內文引用的順序，參考文獻需連續地編號，編號寫在中括號內，如 [1]。參考文獻格式如以下[1]-[6]例子所示。內文引用到參考文獻時，只需簡單地提到參考數字，如[3]，不需寫成“參考文獻 [3]”。

除非有六位作者或更多作者，所有作者的名字都要寫出來，而不要使用“...等”。尚未被發表的論文，即使已經投稿了，仍應標示為“未出版 (unpublished)” [4]。而已經被接受的論文應該標示為“即將出版(in press)” [5]。除了專有名詞和元素符號之外，文章題目的第一個字母需要大寫。

[1] G. Eason, B. Noble, and I. N. Sneddon, “On certain integrals of Lipschitz-Hankel type involving products of Bessel functions,” *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, April 1955.

- [2] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.
- [3] I. S. Jacobs and C. P. Bean, “Fine particles, thin films and exchange anisotropy,” in *Magnetism*, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.
- [4] K. Elissa, “Title of paper if known,” unpublished.
- [5] R. Nicole, “Title of paper with only first word capitalized,” *J. Name Stand. Abbrev.*, in press.
- [6] M. Young, *The Technical Writer's Handbook*. Mill Valley, CA: University Science, 1989.